

PredictionMarket.sol — Flow Diagram

ChainBet · Member 1 Deliverable · APANPS5470 Final Project

1. State Machine

The contract follows a strict four-state lifecycle. Each state transition is irreversible.

State	Allowed Actions	Transition Trigger
OPEN	buyShares, sellShares, getPrice	lockMarket() → LOCKED
LOCKED	None (trading disabled)	resolveMarket() → RESOLVED
RESOLVED	claimWinnings()	All claimed → CLOSED
CLOSED	None	— (terminal)

2. Market Creation (Constructor)

MarketFactory deploys PredictionMarket and seeds initial liquidity into both pools.

Step	Action	Result
1	Factory transfers 2 x initialLiquidity USDC	Contract holds collateral
2	yesPool = noPool = initialLiquidity	Balanced 50/50 odds
3	$k = \text{yesPool} \times \text{noPool}$	AMM invariant set
4	state = OPEN	Trading begins

3. Trading Phase (state = OPEN)

buyShares(bool isYes, uint256 amountIn)

User deposits USDC and receives YES or NO shares. The AMM constant-product formula determines how many shares they get.

Step	Operation	Formula / Detail
1	Pull collateral from buyer	safeTransferFrom(buyer, contract, amountIn)
2	Add to opposite pool	$\text{newPoolIn} = \text{oppositePool} + \text{amountIn}$
3	Rebalance via invariant	$\text{newPoolOut} = k / \text{newPoolIn}$
4	Calculate shares out	$\text{sharesOut} = \text{poolOut} - \text{newPoolOut}$
5	Update user balance	$\text{yesShares}[\text{buyer}] += \text{sharesOut}$
6	Emit event	SharesBought(buyer, isYes, amountIn, sharesOut)

Worked example: Pool = (1000 YES, 1000 NO), k = 1,000,000. User buys YES with 200 USDC.

noPool: $1000 + 200 = 1200$ | yesPool: $1,000,000 / 1200 = 833.33$ | sharesOut = $1000 - 833.33 = \mathbf{166.67 \text{ YES shares}}$

New implied P(YES) = $1200 / (833.33 + 1200) = \mathbf{59.0\%}$ (was 50%)

sellShares(bool isYes, uint256 sharesIn)

Reverse of buyShares. User returns shares to the pool and receives USDC back.

Step	Operation	Formula / Detail
1	Add shares back to pool	$\text{newPoolIn} = \text{samePool} + \text{sharesIn}$
2	Rebalance via invariant	$\text{newPoolOut} = k / \text{newPoolIn}$
3	Calculate collateral out	$\text{amountOut} = \text{oppositePool} - \text{newPoolOut}$
4	Transfer USDC to seller	<code>safeTransfer(seller, amountOut)</code>
5	Emit event	<code>SharesSold(seller, isYes, sharesIn, amountOut)</code>

View Functions (no state change)

Function	Returns	Formula
<code>getYesProbability()</code>	Implied P(YES) in BPS	$\text{noPool} \times 10000 / (\text{yesPool} + \text{noPool})$
<code>getNoProbability()</code>	Implied P(NO) in BPS	$\text{yesPool} \times 10000 / (\text{yesPool} + \text{noPool})$
<code>getPrice(isYes, amountIn)</code>	Preview shares out	Same as buyShares math, read-only
<code>getSellPrice(isYes, sharesIn)</code>	Preview USDC out	Same as sellShares math, read-only

4. Lock Market

When `block.timestamp >= lockTimestamp`, or when oracle/factory calls manually, the market transitions to **LOCKED**. No more `buyShares` or `sellShares` calls are accepted.

Caller	Condition
Anyone	<code>block.timestamp >= lockTimestamp</code>
Oracle	Any time (trusted role)
Factory	Any time (admin role)

5. Oracle Resolution

Only the oracle address can call `resolveMarket(outcome)`. It writes YES or NO on-chain, transitioning the state to **RESOLVED**.

Parameter	Type	Description
<code>outcome</code>	Outcome enum	YES or NO (NONE reverts)

6. Claim Winnings (state = RESOLVED)

Winners call `claimWinnings()` to receive their proportional share of the entire collateral pool, minus a 2% protocol fee.

Step	Operation	Formula / Detail
1	Check state & not yet claimed	<code>require(RESOLVED && !hasClaimed)</code>
2	Identify winning shares	YES won? read <code>yesShares[user]</code>
3	Calculate total pool	<code>totalPool = balanceOf(this) - protocolFees</code>
4	Gross payout	<code>gross = totalPool x userShares / totalWinShares</code>
5	Deduct 2% fee	<code>fee = gross x 200 / 10000</code>
6	Net payout	<code>net = gross - fee</code>
7	Mark claimed	<code>hasClaimed[user] = true</code>
8	Transfer	<code>safeTransfer(user, net)</code>

Worked example: `totalPool` = 3000 USDC, user holds 200 / 500 total YES shares.

`gross` = 3000 x 200/500 = 1200 | `fee` = 1200 x 2% = 24 | **net = 1176 USDC**

Security note: `hasClaimed` is set to true *before* `safeTransfer` — this is the Checks-Effects-Interactions pattern, combined with `ReentrancyGuard` for double protection against reentrancy attacks.

7. Emergency Cancel & Refund

If an event is cancelled (e.g. match postponed), the factory can call `emergencyCancel()`. Users then call `emergencyRefund()` to get back collateral proportional to their total shares held.

Function	Caller	Action
<code>emergencyCancel()</code>	Factory only	Sets state = CLOSED, no winner
<code>emergencyRefund()</code>	Any user	$\text{refund} = \text{pool} \times (\text{yesShares} + \text{noShares}) / \text{grandTotal}$

8. Security Features

Feature	Implementation
Reentrancy protection	OpenZeppelin ReentrancyGuard on all external mutating functions
Safe token transfers	SafeERC20 wraps all IERC20 calls
Checks-Effects-Interactions	State updated before external calls throughout
Access control	<code>onlyOracle</code> , <code>onlyFactory</code> modifiers restrict sensitive functions
Double-claim prevention	<code>hasClaimed</code> mapping prevents repeat withdrawals